

 iamSaikrishna143 / Last_Believe



[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#) [Security](#)

  main [Last_Believe / SeniorLevel.js](#) 





iamSaikrishna143 all update

ce579bf · 9 hours ago



2721 lines (2620 loc) · 125 KB

```
rence between javascript and other programming languages.
in execution context in javascript, event loop works.
rence between let , const and var.
are primitive and non primitive data types?
is the Nullish coalescing operator?
is the arrow function and how is it different from normal function?
is IIFE?
is closures concept in javascript , explain with examples.
is this keyword?
is prototype and prototype inheritance?
rence between map, filter and find method.
is the difference between forEach and map method.
is the callback function? list down the use cases.
is callback hell? How to avoid it.
is a promise , explain with an example.
are promise methods?
is the difference between promise.race and promise.any method.
is event bubbling and event capturing?
is the event delegation concept in javascript?
rence between localStorage, sessionStorage and cookies.

is the immutability concept in javascript?
in the difference between map and set.
is the code splitting concept in javascript.
in memorization concepts in javascript with examples.
is the strict mode in javascript?
rence between call(), bind() and apply() method with example.
is the currying concept in javascript?
rence between shallow and deep copy in object.
in Promise chaining concept with example.
in Object.freeze and Object.seal method.
in design pattern in javascript.
in the concept debouncing and throttling.

question in Reactjs interview
is reactjs and how is it different from other libraries/frameworks?
```

is JSX? What is the difference between JSX and TSX in reactjs.
in the concept of virtual DOM , Shadow DOM in reactjs.
is virtual DOM different from real DOM?
are the different types of components in reactjs?
difference between class and functional component.
what is the state of reactjs?
what are props and the difference between props and states.
what is props drilling and alternative to it?
difference between controlled and uncontrolled components.

when does re-render happen when states change in reactjs?
what happens if you try to update the state directly in reactjs?
how to share data among sibling components?
what is a styled component? Have you implemented theming in reactjs?
what is the react server component?
what are the lifecycle methods of components.
how do functional components handle the lifecycle method of class components?
how to handle side effects in class and functional components?
what are react hooks? List out hooks you have used till date?
difference between useState and useRef hooks in form handling.
what are the differences between useState, useContext and useReducer?
what are the rules of hooks in react?
what are custom hooks? Have you used it in a project, provide an example?
what is a Higher order Component? Explain with examples.
what is the react reconciliation process?
how does react compare virtual DOM with real DOM?
what are context API hooks? Explain with an example?
what is the difference between useCallback and useMemo hooks.
difference between context API and redux.
what is the redux toolkit? How does it simplify redux?
what are middleware in redux?
could you implement a global state without redux?
what is a synthetic event in react?
what is the difference between a synthetic event and a real DOM event?

how to pass arguments to the event handler in react?
how do you handle dynamic routing in react?
how do you manage form validation in react? Explain with examples.
could you implement lazy loading with routes?
how do you optimize forms with hundreds of input fields in react?
could I implement a multistep form with conditional rendering between steps?
what are the keys to react? Why are they important in lists?
what is react.memo and how does it work?
what is the react pure component? explain with roles and use cases.
what is the concept called virtualization in react?
how to optimize rendering of large lists in react?
how to avoid unnecessary re-rendering in react?
what is the purpose of error boundaries in react?
how do you handle errors in asynchronous operations in react?
how do you fetch data in react? explain with examples
what is Axios and how is it different from fetch API?
how do you securely store API keys in react?

ould you implement retries logic for failed API requests in react?

are fragments in react? Why are they used?

is the significance of React.StrictMode?

o you identify and fix the memory leaks in react?

o you build react applications for production?

is the major difference between webpack and babel?

o you configure webpack for react projects?

o you optimize a react application for performance?

in the purpose of .env file in react?

question in NodeJS/ Express JS and MongoDB interview

in the event loop in Node js

in the internal architecture of Node Js

is npm and list out the uses of it.

are the different phases of event loop

in the execution flow of Promise, async/await, setTimeout, setImmediate, process.nextTick()

is the purpose of package.json file in nodejs

is http module and create a server using it

o you create secure server setup using express and https

ill u update any npm existing packages

out some of the global objects in node js

in stream and buffer and their difference

is microtask and macrotask in node js

in piping the data between streams

are the features of express js

happens, if we pass parameter to next function inside middleware

e can jump from one middleware to another skipping some.

is difference between PATCH and PUT API

in the middleware concept in Node js

o create optional path API endpoints in express js

in error handling concept in Express js

JWT and explain the structure of JWT token

an we modify the signature of JWT?

is the authentication and authorization concept ?

o you implement role based nad permission based access control in express js

in in detail best practice of API

in input validation and data sanitization concept in express js

in the concept rate limiting in express js

in the concept of logging and monitoring in node js application

is difference between stateless and stateful API

an we achieve state management in node js application

o you scale node js application using cluster module

is a worker thread ? how does it handle CPU-intensive tasks

in the working with concurrency and parallelism for async task

an you optimise database queries

is the redis database? Explain the uses of it

e can increase the performance of node js application

is the profiling concept in nodejs

is connection pooling concept for database

a middleware for rolebased access control (RBAC) in Node.js

strate the integration of OAuth2.0 for thirdparty authentication in a Node.js app.
ment a refresh token mechanism for JWT authentication in Node.js.
is role of indexes in performance optimization
is the aggregation pipeline in mongoDB? Explain in detail

are transactions? How can we achieve it in mongoDB
ransaction helps in rollback in case of failure
is difference between hashing, encryption and encoding concept
o you identifies slow queries
is replica sets and sharding mechanism in mongodb
o you configure failover and disaster recovery in mongodb
is load balancer and how to implement in nodejs
o avoid schema pitfalls
re care taken to write effcient queries
o implement health checks in mongodb
are the solid principles for software development?
is kafka? Explain the use cases of kafka
an we minimize middleware overhead

questions in the Managerial round as Tech lead/ Sr. Software Engineer
o you ensure your team delivers highquality code within deadlines?
ibe a situation where you had to mentor a junior developer. How did you handle it?
ou explain the SOLID principles and how you apply them in your designs?
in the design pattern in javascript
o you decide which tech stack is suitable for a specifc project?
is difference between development, FT, UAT , preprod and production environment
o you ensure 100% code coverage in your projects?

ou explain the difference between unit testing, integration testing, and functional testing?
o you secure sensitive data in a MERN stack application?
ibe your experience working in an Agile/Scrum environment
in the meetings like sprint planning, daily standup, backlog, sprint review, sprint retrospective 12. Ho
o you onboard new developers to your team and projects?
tools do you use to track project progress and manage tasks?
are the key considerations when designing RESTful APIs?
is your approach to deploying and scaling a MERN stack application?
ould you handle a situation where the application is experiencing a memory leak?
ture you implemented in React is causing performance issues. How would you debug it?
steps would you take if your database suddenly becomes a bottleneck in your application?
ur team is stuck on a critical bug, how would you guide them to a solution?
is your approach to conducting code reviews?
o you optimize MongoDB queries for better performance?
o you manage error handling in an Express.js application?
in how you would secure an Express.js API.
o you optimize React applications for performance?
o you secure a Node.js application?
ould you handle database sharding and replication in MongoDB?

act Concepts

- Understand the limitations of React in large-scale applications?
- How does React manage the Virtual DOM, and what are the benefits?
- Explain the significance of the React Fiber architecture.
- Describe React's reconciliation process and how it updates the DOM efficiently.
- Identify the key differences between functional and class components.

Management & Hooks

- Does it fully replace Redux? Why or why not?
- What are the best practices for managing state in large React apps?
- How do you prevent unnecessary re-renders?
- What are the differences between `useMemo()` and `useCallback()`.
- Explain the use case of `useEffect()` for fetching data.

Finance & Optimization

- Could you optimize large component trees?
- Could you optimize performance for large lists or grids?
- Is lazy loading, and how does it help?

Handling & Debugging

es React handle side effects, and how can you manage them?

ould you handle errors in React, and what is the role of error boundaries?

```

Difference between JavaScript and other programming languages
JavaScript Other Languages (like Java, Python, C++)
Interpreted, dynamically typed Compiled (C, Java), statically typed
Runs in browsers (and Node.js) Usually runs on OS-level compilers
Lightweight, flexible More strict syntax rules
Mainly for web development Used for system, AI, backend, etc.
Event-driven, single-threaded Multi-threaded (Java, C++)
Example:
// JavaScript dynamically infers the type
// Type must be declared

```

on Context & Event Loop

- tion Context: Environment where JS code is executed.
- 1 Execution Context (GEC) - created when the JS program starts.
- ion Execution Context (FEC) - created each time a function is invoked.
- s:
- ion Phase → Memory allocation for variables and functions.
- tion Phase → Code runs line by line.
- Loop: Handles asynchronous code (like setTimeout, fetch).
- tasks from the callback queue to the call stack once it's empty.

```

1:
2:
3: g("1").
4:   ( ) => console.log( 1, 2 );
5: g("3");
6:   1, 3, 2

```

ence between var, let, and const

	scope	Re-declare	Re-assign	Hoisted
var	✓	✓	✓	(undefined)
let	✗	✓	✓	(temporal dead zone)
const	✗	✗	✓	(temporal dead zone)

```

let x = 10;
let x = 20;
let x = 30;

```

Primitive vs Non-Primitive Data Types

Non-Primitive

String, Boolean, Null, Undefined, Symbol, BigInt, Object, Array, Function

```

let x = "Sai"; // primitive
let y = { name: "Sai" }; // non-primitive

```

Nullish Coalescing Operator (??)

Return the right-hand value only if the left-hand value is null or undefined.

```

let name = null ?? "Guest";
console.log(name); // Guest

```

Arrow Function vs Normal Function

Arrow Function	Normal Function
Lexically bound (inherits from parent)	Own this context
Short & clean	Verbose

```

// Arrow Function
const add = (a, b) => a + b;

```

```

// Normal Function
function add2(a, b) {
  return a + b;
}

```

Immediately Invoked Function Expression (IIFE)

A function that runs immediately after it's defined.

```

(function () {
  console.log("IIFE executed!");
})();

```

Closure Concept

A closure gives access to an outer function's scope from an inner function, even after the outer function has executed.

```

function outer() {
  let count = 0;
  function inner() {
    count++;
    console.log(count);
  }
}

```

```

ter = outer();
// 1
// 2

```

keyword

the object that is currently calling the function.

```

:
= {
ai",
{
e.log(this.name);

```

```

()); // Sai

```

ype & Prototype Inheritance

object has a hidden property called `__proto__` that points to another object (its prototype).

```

:
erson(name) {
e = name;

prototype.sayHello = function() {
log("Hello " + this.name);

new Person("Sai");
o(); // Hello Sai
erits from Person.prototype.

```

ence between map, filter, and find

urpose Returns

ransforms array	New array
ilters elements	New filtered array
inds first matching element	Single value

```

:
= [1, 2, 3, 4];
g(arr.map(x => x * 2)); // [2,4,6,8]
g(arr.filter(x => x > 2)); // [3,4]
g(arr.find(x => x > 2)); // 3

```

h vs map

orEach map

ndefined New array

erform side-effects Transform data

```

:
= [1, 2, 3];
h(x => console.log(x * 2)); // Logs only
led = arr.map(x => x * 2); // Returns [2,4,6]

```

Back Function

passed as an argument to another function, to be executed later.

```

// Example
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}

// Usage
greet("John", () => console.log("Welcome!"));

```

Asynchronous handling

Asynchronous APIs (setTimeout, fetch)
 Array methods (map, filter, forEach)

Back Hell

Callbacks that make code hard to read.

```

// Example of Back Hell
function doSomething() {
  // ...
  doSomethingElse(() => {
    // ...
    console.log("Done!");
  });
}

```

Using Promises or async/await:

```

// Example using Promises
function fetchData() {
  // ...
  fetchUser();
  fetchPosts();
  fetchComments();
}

```

Promise (with Example)

A Promise represents the eventual completion (or failure) of an asynchronous operation.

```

// Example of Promise
function fetchData = new Promise((resolve, reject) => {
  // ...
  success = true;
  success) resolve("Data fetched!");
  success) reject("Error occurred");
});

```

```

// Usage
fetchData.then(res => console.log(res))
            .catch(err => console.log(err));

```

More Promise Methods?

Description Example

```

all()    Waits for all promises to resolve or rejects if any fail.  Promise.all([p1,p2])
allSettled() Waits for all promises (fulfilled or rejected). Returns status of each
race()    Returns the first settled (resolve/reject) promise. Promise.race([p1,p2])
any()     Returns the first fulfilled promise (ignores rejections).  Promise.any([p1,p2])
resolve() Converts a value to a resolved promise. Promise.resolve(42)
reject()  Returns a rejected promise. Promise.reject("Error")

```



```

:
1([Promise.resolve(1), Promise.resolve(2)])
s => console.log(res)); // [1,2]

```

Difference between Promise.race() and Promise.any()
 Promise.race() Promise.any()
 First resolved or rejected promise First fulfilled (resolved) promise
 If first promise rejects Only rejects if all promises reject

```

:
new Promise((res) => setTimeout(res, 100, "A"));
new Promise((_, rej) => setTimeout(rej, 50, "Error"));

```

```

ce([p1, p2]); // Rejects with "Error"
y([p1, p2]); // Resolves with "A"

```

Bubbling and Event Capturing

Description Use

Event travels from window → target Rarely used
 Event travels from target → window Commonly used

```

:
querySelector("#child").addEventListener("click", () => console.log("Child"), false); // Bubbling
querySelector("#parent").addEventListener("click", () => console.log("Parent"), true); // Capturing

```

Delegation

where you attach a single event listener to a parent element instead of multiple children.

```

:
querySelector("#list").addEventListener("click", e => {
  if (e.target.tagName === "LI") console.log(e.target.textContent);
});

```

Improves performance and handles dynamically added elements.

LocalStorage vs sessionStorage vs cookies

	localStorage	sessionStorage	cookies
Until manually cleared	Until manually cleared	Until tab/browser closed	Based on expiry date
Limit	~5MB	~5MB	~4KB
Server?	✗	✗	✓
Availability	Browser only	Browser only	Client + Server

Immutability Concept

Immutability means data cannot be modified directly – instead, a new copy is created.

```

:
arr = [1, 2, 3];
newArr = [...arr, 4]; // Creates new array
console.log(arr); // [1,2,3]

```

Difference between Map and Set

	Map	Set
Key-value pairs	Allowed	Unique values
Data type	Not applicable	Not applicable
Entries	Allowed for keys, last one wins ✗	Not allowed

```

:
map = new Map([["a",1],["b",2]]);

```

```
= new Set([1,2,2,3]);
g(map, set); // Map(2), Set(3)
```

Splitting Concept

to split code into smaller chunks so that only required parts load initially – improves performance.

(React / Webpack):

```
module.js').then(module => {
  default();
```

initial load time (used with React.lazy, dynamic imports).

Caching Concept

on technique – stores computed results to prevent re-computation.

```
memoizedAdd() {
  cache = {};
  function(x) {
    if (cache[x]) return cache[x];
    console.log("Calculating...");
    cache[x] = x + 10;
    return cache[x];
  }
}
```

```
const add = memoizedAdd();
console.log(add(5)); // Calculates
console.log(add(5)); // Returns from cache
```

Strict Mode ("use strict")

for, cleaner JavaScript by preventing common mistakes.

```
const x = 10;
let x = 20;
// Error: variable not declared
```

Advantages:

- Prevents accidental globals
- Prevents duplicate parameter names
- Throws more exceptions for debugging

Difference between call(), apply(), and bind()

Invocation	Arguments	Returns
call()	Immediately Comma-separated	Result
apply()	Immediately Array of args	Result
bind()	Later	Any New function

```
const greet = (city) => {
  console.log(`Hi ${this.name} from ${city}`);
}
```

```
const user = { name: "Sai" };
const greet = bind(greet, user);
```

```
greet(user, "Odisha");
greet(user, ["Hyderabad"]);
```

```

tUser = greet.bind(user, "Delhi");
);

```

Currying Concept

a function that takes multiple arguments into a sequence of functions each taking one argument.

```

// Example
function add(a) {
  return function(b) {
    return function(c) {
      return a + b + c;
    }
  }
}

```

```

// Usage
add(2)(3)(4); // 9
// Useful for partial application and code reusability.

```

Shallow vs Deep Copy

Description Example

Shallow Copies only top-level `Object.assign({}, obj)`
 Deep Copies all nested levels `structuredClone(obj)` or `JSON.parse(JSON.stringify(obj))`

```

// Example
const obj = { a: 1, b: { c: 2 } };
const shallow = { ...obj };
const deep = JSON.parse(JSON.stringify(obj));

```

```

// Test
console.log(shallow.b.c); // 99 ❌ (changed)
console.log(deep.b.c); // 2 ✅ (safe)

```

Promise Chaining

Example `.then()` calls to execute asynchronous operations in sequence.

```

// Example
async function getUser() {
  const user = await fetchPosts(user.id)
  const posts = await fetchComments(posts[0].id)
  console.log(posts)
  console.error(err);
}
// Improves readability and flow of async logic.

```

`Object.freeze()` vs `Object.seal()`

	Can Add Props	Can Modify Props	Can Delete Props	Reassign Value
<code>freeze()</code>	❌	❌	❌	❌
<code>seal()</code>	❌	✅	❌	✅

```

// Example
const user = { name: "Sai" };
Object.freeze(user);
user.name = "Krishna"; // ❌ ignored

```

```

const data = { age: 20 };
Object.seal(data);
data.age = 25; // ✅ works
data.name = "Hyd"; // ❌ ignored

```

Patterns in JavaScript

What is a Design Pattern?

Design patterns are reusable solutions to common programming problems. They help to make code more structured, scalable, and maintainable.

JavaScript Design Patterns

Description	Example
Singleton	Ensures only one instance of a class/object exists. Useful for global state (like configuration).
Module	Encapsulates code inside a function (keeps variables private). Used in modern JS modules or IIFE.
Factory	Creates objects without specifying the exact class. Used in UI frameworks, API clients.
Publisher/Subscriber (Pub/Sub)	One-to-many dependency; when one changes, others get updated. Used in event systems, Redux.
Prototype	Creates new objects using an existing object as a blueprint. Core to JS object inheritance.

Example:

Singleton Pattern

```
const AppConfig = (function() {
  let instance;

  function createInstance() {
    return {
      version: "1.0.0", name: "MyApp" };
  }

  return {
    getInstance: function() {
      if (!instance) instance = createInstance();
      return instance;
    }
  };
})();
```

```
const config1 = AppConfig.getInstance();
const config2 = AppConfig.getInstance();
console.log(config1 === config2); // true ✓
```

Module Pattern

```
const CounterModule = (function() {
  let count = 0; // private variable

  return {
    increment: () => ++count,
    decrement: () => --count,
    getCount: () => count
  };
})();

console.log(CounterModule.increment()); // 1
console.log(CounterModule.getCount()); // 1
```

Event Emitter Pattern

```
class EventEmitter {
  constructor() { this.events = {}; }

  on(event, listener) {
    if (!this.events[event]) this.events[event] = [];
    this.events[event].push(listener);
  }
}
```

```
nt, data) {
  is.events[event]) this.events[event].forEach(fn => fn(data));
}
```

```
ter = new EventEmitter();
("greet", name => console.log(`Hello ${name}`));
it("greet", "Sai"); // Hello Sai
```

```
// Pattern
createShape(type) {
  type) {
    circle': return { draw: () => console.log("Drawing Circle") };
    square': return { draw: () => console.log("Drawing Square") };
  }
}
```

```
el = createShape('circle');
w(); // Drawing Circle
```

```
// Type Pattern
on = {
  { console.log(`Hello, I'm ${this.name}`); }
}
```

```
 = Object.create(person);
= "Sai";
(); // Hello, I'm Sai
```

```
// Table
urpose
  One global instance
ncapsulation & privacy
bject creation abstraction
ublish/Subscribe
  Reuse object behavior
```

acing and Throttling

performance optimization techniques that limit how often a function executes – especially useful for scro

```
ing
cution until after a certain time has passed since the last event.
; function runs only once after the user stops triggering the event.
:
ebounce(fn, delay) {
  r;
unction(...args) {
  imeout(timer);
  = setTimeout(() => fn.apply(this, args), delay);
}
```

```
handleSearch = debounce(() => console.log("Search triggered"), 500);
```

Trigger only once after 500ms of no typing

```
document.getElementById("input").addEventListener("keyup", handleSearch);
```

Use: Search boxes, form validation, resize events.

Throttling

Runs the function at regular intervals, no matter how many times the event fires.

But the function runs once every specified time.

Example:

```
function throttle(fn, limit) {
```

```
  let throttle;
```

```
  function(...args) {
```

```
    if (!throttle) {
```

```
      fn.apply(this, args);
```

```
      throttle = true;
```

```
      setTimeout(() => throttle = false, limit);
```

```
handleScroll = throttle(() => console.log("Scrolling..."), 1000);
```

Runs at most once every 1 second

```
document.addEventListener("scroll", handleScroll);
```

Use: Scroll events, button clicks, window resize.

Difference Between Debounce & Throttle

Debounce	Throttle
After user stops triggering	At fixed time intervals
Search input	Scroll or resize
Once after delay	Periodic (limited)
Prevent over-execution	Limit execution rate

What is React.js and how is it different from other libraries/frameworks?

Definition:

React is a JavaScript library developed by Facebook for building user interfaces (UIs), especially single-page

applications. Features:

- Component-based architecture

- Virtual DOM for faster UI updates

- Unidirectional data flow

- Declarative syntax using JSX

Differences from other frameworks

React.js	Angular	Vue.js
Library	Full Framework	Framework
JavaScript / JSX	TypeScript	JavaScript
One-way	Two-way	Two-way
Virtual DOM	Real DOM	Virtual DOM
Shallow	Moderate	Steep
Easy		

Example:

```
componentDidMount() {
```

```
<h1>Hello React!</h1>;
```

JSX? Difference between JSX and TSX

JavaScript XML):

extension that lets you write HTML inside JavaScript.

```
!:
```

```
ent = <h1>Hello, Sai!</h1>;
```

transpiled into:

```
ent = React.createElement("h1", null, "Hello, Sai!");
```

JavaScript React projects, same as JSX but supports type-checking.

```
!:
```

```
= { name: string };
```

```
t = ({ name }: Props) => <h1>Hello, {name}</h1>;
```

JavaScript + XML

TypeScript + XML (adds type safety)

DOM and Shadow DOM

Description

DOM A lightweight in-memory representation of the real DOM used by React to update UI efficiently.

Used in Web Components for encapsulation, keeping styles and logic isolated.

DOM process:

renders a virtual copy of the DOM.

state/props change → React compares (diffing).

the changed elements are updated in the real DOM (reconciliation).

Difference between Virtual DOM and Real DOM

Virtual DOM	Real DOM
Fast	Slower
Updates only changed parts	Re-renders entire tree
In-memory (JS object)	Actual browser DOM
React's UI update logic	Browser's HTML structure

```
!:
```

```
ount + 1);
```

updates only the affected component using Virtual DOM

Types of Components in React.js

Description Example

Component	Simple JS function returning JSX	function App(){return Hi }
Class Component	ES6 class with lifecycle methods	class App extends React.Component { render(){return Hi ;}
Functional Component	Prevents unnecessary re-renders	React.PureComponent
Higher Order Component (HOC)	Function that takes a component and returns a new one	withRouter()
Controlled Component	Input handling based on state or DOM refs	useState vs ref

Difference between Class and Functional Components

Class Component	Functional Component
ES6 class	Simple function
this.state	useState() hook
Lifecycle methods	useEffect()
slightly slower	Faster & simpler
Props Required	Not used

```

import React, { Component } from 'react';

class Hello extends React.Component {
  render() {
    return <h1>Hello {this.props.name}</h1>; }
}

// Functional Component
const Hello = ({name}) => <h1>Hello {name}</h1>;

// State in React.js?
// Definition:
// A built-in object used to store data that changes over time and triggers re-rendering.

// Example:
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <p>Count: {count}</p>
    <button onClick={() => setCount(count + 1)}>Increment</button>
  );
}

// Props are mutable and managed within the component.

// Props vs State: Difference between Props and State
// Properties:
// Passed from parent to child component.
// Immutable: Cannot be changed inside the component.
// Used to make components reusable.

// Example:
function Welcome({ name }) {
  return <h1>Hello {name}</h1>;
}

// Usage:
<Welcome name="Sai" />;

// Props vs State: Comparison
// Props: Passed from Parent to Child, Immutable, Used for communication, Doesn't trigger re-render.
// State: Managed inside Component, Mutable, Manage dynamic data, Triggers re-render.

// Props Drilling and Alternatives
// Problem: Data is passed through multiple nested components unnecessarily.
// Solution:
// 1. Context API
// 2. Redux
// 3. Zustand
// 4. Jotai
// 5. React Query
// 6. Redux Toolkit
// 7. Redux Saga
// 8. Redux Thunk
// 9. Redux Observable
// 10. Redux Form
// 11. Redux DevTools
// 12. Redux DevTools Extension
// 13. Redux DevTools Chrome Extension
// 14. Redux DevTools Redux DevTools Extension
// 15. Redux DevTools Redux DevTools Extension
// 16. Redux DevTools Redux DevTools Extension
// 17. Redux DevTools Redux DevTools Extension
// 18. Redux DevTools Redux DevTools Extension
// 19. Redux DevTools Redux DevTools Extension
// 20. Redux DevTools Redux DevTools Extension

// Props Drilling and Alternatives
// Props Drilling: Data is passed through multiple nested components unnecessarily.
// Alternatives:
// 1. Context API
// 2. Redux
// 3. Zustand
// 4. Jotai
// 5. React Query
// 6. Redux Toolkit
// 7. Redux Saga
// 8. Redux Thunk
// 9. Redux Observable
// 10. Redux Form
// 11. Redux DevTools
// 12. Redux DevTools Extension
// 13. Redux DevTools Chrome Extension
// 14. Redux DevTools Redux DevTools Extension
// 15. Redux DevTools Redux DevTools Extension
// 16. Redux DevTools Redux DevTools Extension
// 17. Redux DevTools Redux DevTools Extension
// 18. Redux DevTools Redux DevTools Extension
// 19. Redux DevTools Redux DevTools Extension
// 20. Redux DevTools Redux DevTools Extension

```



```
UserContext = createContext();
Context.Provider value={name}>
andChild />
rContext.Provider>
Management Libraries - Redux, Zustand, Recoil, MobX.
```

Controlled vs Uncontrolled Components

Controlled	Uncontrolled
React State	DOM itself
via value prop	via ref
Complex forms, real-time validation, form logic	Simple forms

```

const [name, setName] = useState('');



---



```

Re-render happens when state changes in React.js
 Re-renders a component when state or props change.

```

// Controlled
component calls setState() or setCount() (for hooks).
marks the component as "dirty."
React DOM re-renders that component.
React compares old and new Virtual DOM (diffing).
Changed parts are updated in the real DOM (reconciliation).

// Uncontrolled
const [count, setCount] = useState(0);
<button onClick={() => setCount(count + 1)}>+</button>
Click → re-renders component → updates UI efficiently.

```

Warning happens if you try to update the state directly
 Never mutate state directly.

```

// Bad
this.state.count = this.state.count + 1; // doesn't trigger re-render

// Good
this.setState({ count: this.state.count + 1 });

// Functional components
setCount(count + 1);

// Redux
uses setState() to track and schedule updates.
Direct mutation bypasses this mechanism → UI doesn't re-render.

```

How to share data among sibling components
 Can't directly share data (React's data flow is unidirectional).
 Two Solutions:

1. Prop Drilling: Description
 Up → Move state to a common parent, pass as props to both children.
2. Context: Use createContext() for shared global state.

agement Tools Redux, Zustand, MobX, or Recoil.

ks Encapsulate shared logic and reuse it.

(Lift State Up):

```
parent() {
```

```
  name, setName] = useState("Sai");
```

```
  <IdA name={name} />
```

```
  <IdB setName={setName} />
```

is a Styled Component? Have you implemented theming?

ponents is a library for CSS-in-JS, meaning you can style components using JavaScript and template liter

led from "styled-components";

```
on = styled.button`
```

```
  background-color: blue;
```

```
  color: white;
```

```
  padding: 10px;
```

```
  <button>Click Me</Button>
```

```
};
```

ply themes using <ThemeProvider>.

```
};
```

```
const theme = { primary: "blue" };
```

```
  <ThemeProvider theme={theme}>
```

```
    <Primary Button />
```

```
  </ThemeProvider>;
```

```
};
```



main ▾

Last_Believe / SeniorLevel.js

↑ Top

Code

Blame



Raw



er Components (RSCs) allow you to render components on the server, sending pre-rendered HTML to the client in React 18+

```
;
```

es client-side JS bundle size.

es data on the server (no useEffect needed).

r performance and SEO.

```
;
```

Component

```
  default async function UserList() {
```

```
    const users = await fetch("https://api.example.com/users").then(res => res.json());
```

```
    return <ul>{users.map(u => <li>{u.name}</li>)}</ul>;
```

Lifecycle Methods of Class Components

Method Description

constructor() Initialize state and bind methods

componentDidMount() Runs after component renders

componentShouldComponentUpdate() Controls re-render

componentDidUpdate() Runs after state/props update

componentWillUnmount() Cleanup logic (e.g., remove listeners)

```

class:

```

```

import React from 'react';
class App extends React.Component {

```

```

  componentDidMount() { console.log("Mounted"); }

```

```

  componentDidUpdate() { console.log("Updated"); }

```

```

  componentWillUnmount() { console.log("Removed"); }
}

```

Functional Components Handle Lifecycle Methods

Functional components use React Hooks to replace class lifecycle methods.

Method Hook Equivalent

componentDidMount useEffect(() => {...}, [])

componentDidUpdate useEffect(() => {...}, [dependencies])

componentWillUnmount Cleanup inside useEffect

```

class:

```

```

() => {

```

```

  console.log("Mounted or Updated");

```

```

} => console.log("Unmounted");

```

How to Handle Side Effects

Side effects = anything that affects something outside the function (API calls, subscriptions, timers, etc.)

Type Method

Functional componentDidMount, componentDidUpdate, componentWillUnmount

Class useEffect() hook

```

// Functional:

```

```

() => {

```

```

  fetch('api/data').then(...);
}

```

```

// Class:

```

```

class App {
  componentDidMount() {

```

```

    fetch('api/data').then(...);
  }
}

```

What are React Hooks? List Hooks You've Used

React Hooks allow you to use state and lifecycle features in functional components (introduced in React 16.8).

Commonly Used Hooks:

useState

useEffect

useContext

useReducer

useRef Access DOM elements or persist data

useMemo Memoize computed values

useCallback Memoize functions

useLayoutEffect Manage complex state

useEffect Runs before paint for sync updates

iveHandle Customize refs in forwardRef components

ence between useState and useRef in Form Handling

seState useRef

e-renders component when value changes Doesn't trigger re-render

eactive data Mutable reference

tate variable ref.current

controlled input Uncontrolled input

!:

e - Controlled Component

e, setName] = useState('');

ue={name} onChange={e => setName(e.target.value)} />;

- Uncontrolled Component

tRef = useRef();

={inputRef} />;

g(inputRef.current.value);

ence between useState, useContext, and useReducer

se When to Use Example

manage simple local state When state logic is simple and independent const [count, setCount] = useStat

Share state globally across components When multiple components need the same data const value = use

Manage complex state logic (like Redux) When multiple states depend on one another const [state, dis

!:

cer = (state, action) => {

ction.type) {

INCREMENT": return { count: state.count + 1 };

t: return state;

te, dispatch] = useReducer(reducer, { count: 0 });

of Hooks in React

special functions with strict rules:

ly call hooks at the top level (not inside loops, conditions, or nested functions).

!:

fect(() => {}, []);

ect:

ndition) useEffect(() => {}, []);

ly call hooks inside React function components or custom hooks.

!:

ion MyComp() { useState(0); }

ook names must start with use (React uses this to validate hooks).

aintain the same hook order on every render.

re Custom Hooks?

ook is a reusable function that uses built-in hooks (useState, useEffect, etc.) to extract and reuse log

- Custom Hook for Window Width:

seWindowWidth() {

idth, setWidth] = useState(window.innerWidth);

t(() => {

handleResize = () => setWidth(window.innerWidth);

```
.addEventListener("resize", handleResize);
() => window.removeEventListener("resize", handleResize);
```

```
idth;
```

```
component
```

```
h = useWindowWidth();
```

```
g(width);
```

```
ie: API fetching logic, form handling, debouncing, authentication, etc.
```

```
is a Higher-Order Component (HOC)?
```

```
function that takes a component and returns a new component with additional functionality.
```

```
es:
```

```
withLogger(WrappedComponent) {
```

```
function Enhanced(props) {
```

```
  e.log("Props:", props);
```

```
  <WrappedComponent {...props} />;
```

```
utton(props) {
```

```
  button>{props.label}</button>;
```

```
edButton = withLogger(Button);
```

```
ies:
```

```
ntication wrappers
```

```
ng, analytics
```

```
reuse before hooks existed
```

```
is the React Reconciliation Process?
```

```
tion is React's diffing algorithm that determines how the Virtual DOM and Real DOM should sync after a c
```

```
creates a new Virtual DOM tree after state/prop change.
```

```
res (diffs) new tree with the old one.
```

```
minimal differences.
```

```
es only changed elements in the Real DOM.
```

```
cept: "React Fiber" – an optimized reconciliation algorithm for smoother rendering.
```

```
React Compares Virtual DOM with Real DOM
```

```
ription
```

```
Virtual DOM React builds an in-memory tree of components
```

```
gorithm Compares new Virtual DOM with previous one
```

```
Updates Only changes are applied to the Real DOM (batch updates)
```

```
Phase Real DOM is updated efficiently
```

```
es:
```

```
ne list item → React only updates that node, not the entire list.
```

```
are Context API Hooks? Explain with an Example
```

```
It provides a way to share global data without passing props manually down every level.
```

```

createContext() → Create context
useContext() → Consume context
//
const ThemeContext = createContext();

```

```

App() {

```

```

  <Context.Provider value="dark">
    <Toolbar />
    <ThemeContext.Provider>

```

```

    <Toolbar /> {
      theme = useContext(ThemeContext);
    } <p>Theme is {theme}</p>;
  }

```

use: Theme, Language, Authentication, User Info

Difference between useCallback and useMemo

useCallback Returns Use Case

```

useCallback(fn, deps)    Memoizes a function A memoized function Prevents re-creation of functions
useMemo(value, deps)     Memoizes a value A memoized value Prevents re-computation of expensive values
//

```

```

const memoizedFn = useCallback(() => doSomething(a, b), [a, b]);
const memoizedValue = useMemo(() => compute(a, b), [a, b]);
//
In performance optimization to avoid unnecessary re-renders.

```

Difference Between Context API and Redux

Context API Redux

Share global state Manage complex global state

Basic (lightweight) Predictable + structured

Minimal More setup

Re-renders entire tree Uses selective subscription

✗ Not available ✓ Redux Thunk, Saga

✗ Limited ✓ Excellent debugging tools

Context → Small apps

Redux → Large-scale apps with complex logic.

What is Redux Toolkit (RTK)? How It Simplifies Redux

Redux Toolkit (RTK) is the official, recommended way to write Redux logic – it simplifies setup, reduces boilerplate.

```

import { configureStore, createSlice } from "@reduxjs/toolkit";

```

```

const counterSlice = createSlice({
  name: "counter",
  initialState: 0,
  reducers: {
    increment: (state) => state + 1,
  },
});

```

```

st { increment } = counterSlice.actions;
e = configureStore({ reducer: { counter: counterSlice.reducer } });
is Better:
Immer (you can "mutate" state safely).
uilt-in Thunk middleware for async actions.
e configuration with configureStore().
es boilerplate like action creators & switch statements.
nts, routing, and form handling – explained clearly, practically, and with examples 📌

```

What are Middleware in Redux?

A middleware in Redux is a function that sits between dispatching an action and the reducer – used to handle side effects. Flow:

Middleware → Reducer → Store

Example: Logging Middleware

```

const loggerMiddleware = store => next => action => {
  log("Dispatching:", action);
  return next(action);
};

```

Common Middlewares:

- thunk → Handle async operations
- saga → Handle complex async logic via generators
- logger → Log actions and states

How would you implement a global state without Redux?

Approach 1 – React Context API

```

const GlobalContext = createContext();

```

```

function App() {
  const [user, setUser] = useState(null);

```

```

  return (
    <GlobalContext.Provider value={{ user, setUser }}>

```

```

      <div>

```

```

        <GlobalContext.Provider>

```

```

        <div>
          <button onClick={handleClick}>Click</button>

```

Approach 2 – Zustand or Recoil

These are state management libraries for global state, simpler than Redux.

Approach 3 – Custom Hooks + Context

Encapsulate the logic inside a custom hook and share through context.

What is a Synthetic Event in React?

A Synthetic Event is React's wrapper around the browser's native event system – provides cross-browser compatibility. Example:

```

function handleClick(e) {
  console.log(e.type); // Synthetic event
}

```

```

<button onClick={handleClick}>Click</button>

```

event system pools and reuses events for performance.

Difference between Synthetic Event and Real DOM Event

Synthetic Event	Real DOM Event
React	Browser
Consistent	Cross-browser consistent
Uses event pooling	Browser dependent
Native execution	
React event handlers (onClick, onChange)	Vanilla JS (addEventListener)
button onClick={fn}>	element.addEventListener("click", fn)

Pass Arguments to Event Handlers in React

1 - Inline Arrow Function

```
Click={() => handleClick(5)}>Click</button>
```

2 - Using bind()

```
Click={handleClick.bind(this, 5)}>Click</button>
```

```
`:
```

```
handleClick(id) {
```

```
  log("Clicked:", id);
```

Handle Dynamic Routing in React

Router DOM v6+ for dynamic routes.

```
`:
```

```
path="/user/:id" element={<User />} />
```

```
useParams } from "react-router-dom";
```

```
useRouter() {
```

```
  const { id } = useParams();
```

```
  return <h2>User ID: {id}</h2>;
```

Dynamic routes help display content based on URL parameters (/user/1, /user/2, etc.).

Manage Form Validation in React

1 - Manual Validation

```
const [email, setEmail] = useState("");
```

```
const [error, setError] = useState("");
```

```
const handleSubmit = (e) => {
```

```
  e.preventDefault();
```

```
  if (!email.includes("@")) setError("Invalid email");
```

2 - Using Libraries

Formik → Handles complex form states.

Yup → Schema-based validation.

React Hook Form → Fastest & minimal re-renders.

Example (React Hook Form):

```
import { useForm } from "react-hook-form";
```


progress bar or conditional rendering for optional steps.

are Keys in React? Why are they Important in Lists?

Unique identifiers given to elements in a list to help React efficiently update and render list items.

Ex:

```
(user => (
  {user.id}>{user.name}</li>
```

Important:

React identify which items changed, added, or removed.

Avoids unnecessary re-renders.

Keeps component state consistent during updates.

Avoid using index as key (except for static lists):

```
index}></li> // bad for dynamic lists
```

What is React.memo and How Does It Work?

React.memo is a higher-order component (HOC) that memoizes functional components – it prevents re-rendering unless

Ex:

```
MyComponent = React.memo(function MyComponent({ name }) {
  log("Rendered");
  <p>{name}</p>;
```

How to Use:

Wrap pure functional components.

Prevents component re-renders often with same props.

How it Works:

Compares previous props with new props using shallow comparison – if equal, it skips rendering.

What is a React Pure Component?

React.PureComponent is a class component that implements shouldComponentUpdate() automatically with a shallow prop & state

Ex:

```
MyComponent extends React.PureComponent {
  {
    e.log("Rendered");
    <p>{this.props.name}</p>;
```

How it Works:

Prevents unnecessary re-renders when props/state do not change.

Similar to React.memo but for class components.

Warning: Not for deep objects – shallow comparison won't detect nested changes.

What is the Concept Called Virtualization in React?

Windowing (or virtualization) means rendering only a small portion of a large list visible on the screen – not the

Use Case: Performance optimization for large data (e.g., 10,000+ items).

How to use (using react-window):

```
import {FixedSizeList as List} from "react-window";
```

```
const list = {
  itemCount: {10000}
  itemSize: {35}
  width: {300}>
```

```
(x, style) => <div style={style}>Item {index}</div>
```

Notes:

es DOM nodes.

ves scroll performance.

o You Optimize Rendering of Large Lists in React?

ques:

alization - Using libraries like react-window or react-virtualized.

ation / Infinite Scroll - Fetch data in chunks.

zation - Use React.memo, useMemo, or useCallback.

ptimization - Unique keys help React track items.

Inline Functions / Objects - Prevent unnecessary prop changes.

!:

```
= React.memo(({ item }) => <li>{item}</li>);
```

o Avoid Unnecessary Re-rendering in React

ques:

olution

ging unnecessarily Use React.memo or PureComponent

recreated on each render Use useCallback

computations Use useMemo

ext updates Split contexts or use selective providers

tate updates Combine state updates, use batching

!:

```
leClick = useCallback(() => console.log("Click"), []);
```

Is the Purpose of Error Boundaries in React?

aries are React components that catch runtime errors in their child components and display a fallback U

!:

```
rBoundary extends React.Component {
```

```
{ hasError: false };
```

```
etDerivedStateFromError() { return { hasError: true }; }
```

```
tDidCatch(error, info) { console.log(error, info); }
```

```
{
```

```
is.state.hasError) return <h2>Something went wrong.</h2>;
```

```
this.props.children;
```

re:

UI components (like dynamic or third-party widgets).

o You Handle Errors in Asynchronous Operations in React?

ch 1 - Try/Catch

```
tion fetchData() {
```

```
res = await fetch("/api");
```

```
data = await res.json();
```

```
(error) {
```

```
e.error(error);
```

ch 2 - Error State

```
or, setError] = useState(null);
```

```
() => {
  api")
  res => res.json()
  (err => setError(err.message));
}
```

h 3 – Error Boundary for Async Components (React 18+)

se or boundary for network errors.

o You Fetch Data in React? (With Examples)

```
fetch()
() => {
  https://jsonplaceholder.typicode.com/users")
  res => res.json()
  setUsers)
  (console.error);
}
```

async/await

```
() => {
  adData = async () => {
```

```
    t res = await fetch("/api");
    t data = await res.json();
    ata(data);
    h (e) {
    ole.error(e);
  }
}
```

```
();
```

axios

```
os from "axios";
```

```
() => {
  t("/api/data").then(res => setData(res.data));
}
```

libraries

Query / TanStack Query

or caching, retry, and background fetching.

s Axios and How is it Different from Fetch API?

axios Fetch API

nal library Native browser API

sponse Auto JSON parsing Must call res.json()

ling Rejects on 4xx/5xx Must manually check res.ok

ncelation  Supports via tokens  Manual workaround

 Built-in support  Requires custom code

rs  Easy to set global headers  Not built-in

gress  Built-in  Needs extra handling

```
};
```

```
"/users")
```

```
s => console.log(res.data))
```

```
err => console.error(err));
}
// Feature-rich and convenient, while fetch() is lightweight and native.

// How do you securely store API keys in React?
// Since JavaScript runs in the browser, so you can't fully hide secrets in frontend code.
// Best practices:
// 1. Store sensitive keys on a backend server and call the backend instead.
// 2. For non-sensitive keys (like Firebase public keys), use .env files:
// .env
APP_API_KEY=your_api_key
// Access via:
import { REACT_APP_API_KEY } from '@env';
// Commit .env files to GitHub; add them to .gitignore.
```

```
// How would you implement retry logic for failed API requests in React?
// You can retry API requests using:
// 1. Custom logic with Axios interceptors
// 2. Libraries like react-query or axios-retry
// Here's an example using Axios retry:
import axios from "axios";
import axiosRetry from "axios-retry";

// Configure axios with retry logic
const axiosInstance = axios.create({
  // ...
});
axiosRetry(axiosInstance, {
  retries: 3,
  retryDelay: axiosRetry.exponentialDelay
});

// Usage
axiosInstance.get("/api/data")
  .then(res => console.log(res.data))
  .catch(err => console.error("Failed after 3 retries", err));
```

How are fragments in React? Why are they used?

Fragments allow grouping multiple elements without adding extra DOM nodes.

```
<div>
  <h1>
  <p>Description</p>
</div>
```

This avoids unnecessary <div> wrappers and keeps the DOM clean.

```
// What is the significance of React.StrictMode?
// React.StrictMode helps highlight potential problems during development.
// It checks for:
// 1. Unsafe lifecycle methods
// 2. Unsafe APIs
// 3. Side effects in rendering
// 4. These warnings do not affect production builds.
// Usage:
import { StrictMode } from 'react';
// ...
<StrictMode>
  <App />
</StrictMode>
```

How do you identify and fix memory leaks in React?

Common causes include unmounted components still holding event listeners or subscriptions.

operations not cancelled.

cleanup in useEffect:

```
() => {
  timer = setInterval(() => console.log("Running..."), 1000);
  () => clearInterval(timer); // cleanup
}
```

tools like Chrome Performance tab or React Profiler to detect leaks.

How do you build React applications for production?

How to build

How does Webpack process JS and CSS

How does Webpack process images and bundles via Webpack

How does Webpack process static files in the /build folder

How does Webpack deploy the build to services like Netlify, Vercel, or AWS S3.

What is the major difference between Webpack and Babel?

Webpack vs Babel

Webpack: Module bundler, JavaScript compiler

Babel: Transpiles ES6+ → ES5 for browser compatibility

Webpack combines files, Converts arrow functions to normal functions

Often used together: Webpack bundles, Babel transpiles.

How do you configure Webpack for React projects?

Webpack dependencies:

npm install webpack webpack-cli babel-loader @babel/core @babel/preset-react html-webpack-plugin --save-dev

webpack.config.js:

```
module.exports = {
  entry: './src/index.js',
  output: { filename: 'bundle.js', path: __dirname + '/dist' },
  module: {
    rules: [{ test: /\.js$/, exclude: /node_modules/, use: 'babel-loader' }]
  }
}
```

plugins: [new HtmlWebpackPlugin({ template: './public/index.html' })],

.babelrc file:

```
{
  "presets": ["@babel/preset-env", "@babel/preset-react"]
}
```

How do you optimize a React application for performance?

Use React.memo to prevent unnecessary re-renders.

Use useCallback and useMemo to cache expensive functions/values.

Lazy load components and routes.

Split code using dynamic imports.

Use React Profiler to identify slow components.

Minimize re-renders by avoiding state in parent components unnecessarily.

What is the purpose of .env file in React?

The .env file stores environment-specific variables like API URLs or keys.

Common files: .env.development and .env.production for different stages.

bles must start with REACT_APP_.

_APP_API_URL=https://api.example.com

s in React:

```
apiUrl = process.env.REACT_APP_API_URL;
```

the event loop in Node.js

js runs on a single-threaded, non-blocking event loop that allows it to handle many concurrent requests. The event loop constantly checks for pending tasks (I/O, timers, promises, etc.) and executes callbacks asynchronously.

```
log('Start');
```

```
((() => console.log('Timer'), 0);
```

```
log('End');
```

The event loop places setTimeout callback in the callback queue, which runs after the main thread finishes.

the internal architecture of Node.js

Architecture has:

V8 Engine - Compiles JavaScript into machine code.

- Handles the event loop, I/O, and async operations.

Queue - Holds callbacks waiting for execution.

Thread Pool - Handles expensive operations (like file read, DNS, crypto).

Bindings - Connect JS with C++ core modules.

Architecture = JS (V8) + Libuv (C++) + Event loop → Asynchronous Non-blocking architecture.

npm and list out the uses of it

Node Package Manager) is used to manage Node.js packages.

is installed automatically with Node.js.

Install external libraries (npm install express)

Define project dependencies (package.json)

Install custom packages

Run scripts (npm start, npm test)

Explain the different phases of event loop

The event loop has 6 phases:

1. Timer Phase - Executes callbacks from setTimeout() and setInterval().

2. Pending Callbacks - Executes I/O callbacks deferred to the next loop.

3. Prepare - Used internally by Node.

4. Poll Phase - Retrieves new I/O events, executes their callbacks.

5. Check Phase - Executes setImmediate() callbacks.

6. Close Callbacks - Executes close event callbacks (e.g., socket close).

Explain the execution flow of Promise, async/await, setTimeout, setImmediate, process.nextTick()

Execution order (synchronous → microtasks → macrotasks):

JavaScript Code → process.nextTick() → Promises → setTimeout() → setImmediate()

```
log('A');
```

```
((() => console.log('B'), 0);
```

```
setTimeout(() => console.log('C'));
```

```

xtTick(() => console.log('D'));
solve().then(() => console.log('E'));
g('F');

```

n:

ss.nextTick() and Promises are microtasks, executed before timer and I/O callbacks.

the purpose of package.json file in Node.js
 etadata file that defines the Node.js project:
 ct name, version, description
 dependencies (dependencies / devDependencies)
 ts (npm start, npm test)
 file (main)

```

"myapp",
": "1.0.0",
"index.js",
": { "start": "node index.js" },
ncies": { "express": "^4.18.2" }

```

HTTP module and create a server using it
 HTTP module in Node.js allows you to create a web server that handles HTTP requests/responses.

```

= require('http');

er = http.createServer((req, res) => {
eHead(200, { 'Content-Type': 'text/plain' });
'Hello from Node.js Server!');

ten(3000, () => console.log('Server running on port 3000'));

```

you create a secure server setup using Express and HTTPS
 e the https core module along with Express.

```

require('fs');
s = require('https');
ess = require('express');

= express();

teServer({
readFileSync('private-key.pem'),
.readFileSync('certificate.pem')

```



```
3, () => console.log('Secure server running on port 443'));

', (req, res) => res.send('Secure HTTPS Server')));
```

1 you update any npm existing packages

date a single package:

pdate package-name

date all packages:

pdate

stall a specific version:

nstall package-name@latest

eck outdated packages:

utdated

ut some of the global objects in Node.js

bal objects:

name → directory name of current module

ename → file name of current module

le → for logging

ss → provides info about current process

r → handles binary data

meout, setInterval, setImmediate → timers

l → similar to window in browsers

n Stream and Buffer and their Difference

r: Temporary memory storage for binary data, used to handle raw data efficiently.

m: Handles continuous flow of data, can read/write large files efficiently without loading entire conten

```
require('fs');
able = fs.createReadStream('input.txt'); // stream
able = fs.createWriteStream('output.txt');
ipe(writable); // piping data from stream
:
uffer Stream
ing Holds entire data in memory Handles data in chunks
ge High for large files Low, efficient
mall files, binary data Large files, real-time data (video/audio)
```

s Microtask and Macrotask in Node.js

tasks: High priority, executed after current operation but before the event loop continues.

les: process.nextTick(), Promise.then()

tasks: Executed in event loop phases like timers or I/O.

les: setTimeout(), setInterval(), setImmediate()

order example:

```
g('Start');
(() => console.log('Timeout'), 0);
xtTick(() => console.log('NextTick'));
solve().then(() => console.log('Promise'));
g('End');
```

Streaming Data Between Streams

allows connecting a readable stream to a writable stream.
efficiently transfers large data without loading it all into memory.

```
const fs = require('fs');
const readStream = fs.createReadStream('input.txt');
const writeStream = fs.createWriteStream('output.txt');

readStream.pipe(writeStream);

// Use cases: File copy, video streaming, server responses.
```

Key Features of Express.js

- Minimal and fast web framework for Node.js
- Routing capabilities (app.get, app.post)
- Middleware support for request processing
- Template engines support (e.g., Pug, EJS)
- JSON handling of JSON, URL-encoded requests
- Static file handling & static file serving
- Rich ecosystem of plugins & modules

Error Handling in Express.js Middleware

Passing a parameter to next(err) signals an error.
Express will skip remaining middleware and go to error-handling middleware.

```
app.use((req, res, next) => {
  // ...
  const err = new Error('Something went wrong');
  next(err); // goes to error handler
});
```

Skipping Middleware in Express.js

Can use next('route') in route-level middleware to skip remaining middlewares for that route:

```
app.use('/user', middleware1, middleware2, (req, res) => res.send('Final'));

// middleware1
function middleware1(req, res, next) {
  // ...
  skip() next('route'); // skips middleware2
}

// middleware2
function middleware2(req, res, next) {
  // ...
}
```

Differences Between PATCH and PUT API

PUT PATCH

Replaces entire resource Updates partial resource



PUT: Full object PATCH: Only fields to update

PUT /user/1 {name:"A", age:20} PATCH /user/1 {age:21}

Understanding the Middleware Concept in Node.js

Middleware is a function that executes during request-response cycle.
perform tasks like logging, authentication, parsing, error handling.

```
app.use((req, res, next) => {
  // ...
});
```

```
log(`${req.method} ${req.url}`);
```

```
:
```

```
cation-level
```

```
r-level
```

```
-handling middleware
```

```
-in middleware
```

```
-party middleware
```

```
› Create Optional Path API Endpoints in Express.js
```

```
in route parameters for optional paths:
```

```
user/:id?', (req, res) => {
```

```
arams.id) res.send(`User ID: ${req.params.id}`);
```

```
.send('All users');
```

```
› All users
```

```
.23 → Specific user
```

```
› Error Handling Concept in Express.js
```

```
-handling middleware has 4 arguments: (err, req, res, next)
```

```
atches errors passed by next(err) or thrown in async code.
```

```
rr, req, res, next) => {
```

```
error(err.stack);
```

```
us(500).send({ error: err.message });
```

```
› Practices:
```

```
sync wrapper functions for promises
```

```
› log errors
```

```
› proper HTTP status codes (400, 404, 500)
```

```
› JWT and Explain the Structure of JWT Token
```

```
Web Token) is a compact, URL-safe token format used for authentication and information exchange.
```

```
load.signature
```

```
r: Token type and algorithm
```

```
HS256", "typ": "JWT" }
```

```
ad: Contains user data/claims
```

```
3, "role": "admin" }
```

```
ture: Encoded header + payload + secret
```

```
(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)
```

```
› e: Authenticate users without storing sessions on the server (stateless).
```

```
› Can We Modify the Signature of JWT?
```

```
› cannot modify the signature without invalidating the token.
```

```
› Integrity is guaranteed by the signature:
```

```
› If you tamper with header or payload, signature will not match → token rejected.
```

```
› To "change" data, generate a new token using the secret/private key.
```

```
› Authentication and Authorization Concept?
```

```
› Authentication: Verify identity of user (login, JWT, OAuth)
```

```
› Example: Login with email/password → issue token
```

Authorization: Verify what the user can access

Example: Admin can delete posts, regular user cannot

Flow: Authentication → Authorization

How to Implement Role-Based and Permission-Based Access Control in Express.js

Middleware:

```
function authorizeRoles(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ message: "Forbidden" });
    }
    next();
  };
}
```

```
router.delete('/user/:id', authorizeRoles('admin'), (req, res) => {
  // ...delete logic
  res.json({ message: 'User deleted' });
});
```

Role-based: Check user roles (admin, editor)

Permission-based: Check granular permissions (canDelete, canEdit)

Additional Best Practices for API

Use proper HTTP methods: GET, POST, PUT, PATCH, DELETE

Return status codes correctly: 200, 201, 400, 401, 403, 404, 500

Use versioned APIs: /api/v1/users

Input validation & sanitization: Avoid injection attacks

Authentication & Authorization: JWT/OAuth

Rate limiting & throttling: Prevent abuse

Consistent response handling: Standardized JSON responses

Logging & monitoring: Track errors and performance

HTTPS: Secure API communication

Pagination & filtering: For large datasets

How to Implement Input Validation and Data Sanitization in Express.js

Validation: Check input meets requirements (email format, password length)

Sanitization: Clean input to prevent malicious content (XSS, SQL injection)

Using express-validator:

```
const { body, validationResult } = require('express-validator');

router.post('/register',
  body('email').isEmail(),
  body('password').isLength({ min: 6 }),
  (req, res) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) return res.status(400).json({ errors: errors.array() });

    // ...register logic
    res.json({ message: 'User registered' });
  });
```

How to Implement Rate Limiting in Express.js

Why: Prevent abuse and DDoS attacks

```

ries: express-rate-limit
limit = require('express-rate-limit');

ter = rateLimit({
: 15 * 60 * 1000, // 15 mins
// limit per IP

api/', limiter);

```

Logging and Monitoring in Node.js Application

ng: Track requests, errors, and events

ries: winston, morgan, pino

oring: Track app performance, uptime, errors

: PM2, New Relic, Datadog, Sentry

actice: Log both info (requests) and error (exceptions) with timestamps.

Difference Between Stateless and Stateful API

Stateless API Stateful API

ps state ✗ No ✓ Yes

EST APIs SOAP, traditional sessions

y Easy Harder to scale

tion JWT or token Sessions/cookies

icroservices, mobile apps Legacy systems, in-memory sessions

How We Achieve State Management in Node.js Application

ns for state management:

emory storage: global object or Map (simple apps)

ases: Redis, MongoDB, PostgreSQL for persistent state

ons: express-session (stateful)

```
ion = require('express-session');
```

```

ssion({
'mySecret',
false,
itialized: true

```

-based state: Store user state in JWT payload (stateless)

actice: Use stateless JWT + database combination for scalability.

How You Scale Node.js Application Using Cluster Module

js runs single-threaded by default, but Cluster module allows creating child processes (workers) that sh
worker can handle requests in parallel, improving CPU utilization.

```

ter = require('cluster');
= require('http');
PUs = require('os').cpus().length;

r.isMaster) {
log(`Master ${process.pid} is running`);
i = 0; i < numCPUs; i++) cluster.fork();
on('exit', (worker) => console.log(`Worker ${worker.process.pid} died`));

```

```

createServer((req, res) => res.end('Hello from Node.js')).listen(3000);
log(`Worker ${process.pid} started`);

```

• Utilize multi-core CPUs and improve throughput.

What is a Worker Thread? How Does It Handle CPU-Intensive Tasks

Worker threads allow running CPU-heavy tasks in a separate thread without blocking the main event loop. Ideal for computations, image processing, or crypto.

```

const Worker = require('worker_threads');

// Worker thread
const worker = new Worker('worker.js');

// Main thread
const parentPort = require('worker_threads').parentPort;
let sum = 0;
for(let i=0; i<1e9; i++){sum+=i;}
parentPort.postMessage(sum);
parentPort.on('message', console.log);
// Main thread stays responsive while CPU-intensive tasks run in parallel.

```

Concurrency vs Parallelism for Async Tasks

Concurrency: Multiple tasks progress overlapping in time (non-blocking I/O)

Parallelism: Multiple tasks executed simultaneously on different threads/cores

Concurrency with Promise.all():

```

async function fetchAll() {
  const results = await Promise.all([fetch('/api1'), fetch('/api2')]);
  log(results);
}

```

Notes:

• Event loop → concurrency for I/O

• Worker threads → parallelism for CPU tasks

How Can You Optimize Database Queries

• Indexes on frequently queried columns

• Limit fields and rows in SELECT (SELECT id, name FROM users)

• Pagination for large datasets

• Batch N+1 queries → Use JOIN or aggregation

• Caching (Redis or in-memory cache)

• Prepared statements to prevent repeated parsing and SQL injection

What is the Redis Database? Explain Its Uses

Redis is an in-memory key-value store (NoSQL), extremely fast.

• Session storage

• Caching frequently accessed data

• Pub messaging

• Rate limiting

• Leaderboards and counters

```

const redis = require('redis');
const client = redis.createClient();

client.set('user:1', JSON.stringify({name: 'Alice'}));

client.get('user:1', (err, data) => console.log(JSON.parse(data)));

```

```
Can Increase the Performance of Node.js Application
Cluster module or PM2 for multi-core scaling
Asynchronous programming → Avoid blocking I/O
Cache frequently accessed data (Redis, memory)
Optimize database queries and use indexes
Code splitting and lazy loading for large apps
Gzip compression for API responses (compression middleware)
Minimize middleware overhead and heavy synchronous operations
```

Node.js Profiling Concept in Node.js

```
Profiling = Measuring CPU, memory, and performance bottlenecks.
Tools:
- Node.js built-in profiler: node --inspect index.js
- Chrome DevTools: Attach Node process for CPU profiling
- pm2-profiler / PM2 profiling tools
Goal: Identify memory leaks, long-running operations, or bottlenecks.
```

Node.js Connection Pooling Concept for Database

```
A connection pool maintains a set of reusable DB connections to improve performance.
Avoids creating a new connection per query.
```

```
MySQL:
```

```
const mysql = require('mysql2');
const pool = mysql.createPool({
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: 'test',
  database: 'test',
  connections: true,
  poolLimit: 10
});
```

```
pool.query('SELECT * FROM users', (err, results) => console.log(results));
// Benefit: Faster DB queries and efficient resource usage.
```

Middleware for Role-Based Access Control (RBAC)

```
const rbac = (allowedRoles) => {
  return (req, res, next) => {
    const { userRole } = req.user; // assume user is attached by auth middleware
    if (!allowedRoles.includes(userRole)) {
      return res.status(403).json({ message: "Forbidden" });
    }
    next();
  };
};
```

```
app.get('/admin/data', rbac(['admin']), (req, res) => res.send('Deleted'));
```

Strategy Integration of OAuth2.0 for Third-Party Authentication

```
Using Passport.js for Google OAuth2:
const passport = require('passport');
const GoogleStrategy = require('passport-google-oauth20').Strategy;
```

```

se(new GoogleStrategy({
  : 'GOOGLE_CLIENT_ID',
  cret: 'GOOGLE_SECRET',
  URL: "/auth/google/callback"
Token, refreshToken, profile, done) => {
  user in DB
  l, profile);

  auth/google', passport.authenticate('google', { scope: ['profile', 'email'] }));
  auth/google/callback',
  .authenticate('google', { failureRedirect: '/login' })),
  s) => res.redirect('/dashboard')

: Secure, token-based third-party authentication without handling passwords directly.

```

Implement a Refresh Token Mechanism for JWT Authentication in Node.js

```

s Token: Short-lived JWT for authentication
sh Token: Long-lived token to get a new access token
tion Example:
= require('jsonwebtoken');

e Access Token
enerateAccessToken(user) {
wt.sign({ id: user.id, role: user.role }, process.env.ACCESS_SECRET, { expiresIn: '15m' });

e Refresh Token
enerateRefreshToken(user) {
wt.sign({ id: user.id }, process.env.REFRESH_SECRET, { expiresIn: '7d' });

token endpoint
/token', (req, res) => {
token } = req.body;
en) return res.sendStatus(401);
fy(token, process.env.REFRESH_SECRET, (err, user) => {
r) return res.sendStatus(403);
accessToken = generateAccessToken({ id: user.id, role: user.role });
on({ accessToken });

: Enhances security by short-lived access tokens while avoiding repeated logins.

```

Indexes in Performance Optimization

```

es speed up queries by allowing MongoDB to find documents faster.
ut indexes → full collection scan (slow)
of indexes:
e field: {name: 1}
und: {name: 1, age: -1}

```


hashed, geospatial

```
createIndex({ email: 1 }, { unique: true });
```

• Faster queries, efficient sorting, and reduced CPU usage.

Aggregation Pipeline in MongoDB

to process data in stages, like SQL's GROUP BY and JOIN.
Stages: \$match, \$group, \$project, \$sort, \$limit

```
aggregate([
  { $match: { status: 'completed' } },
  { $group: { _id: "$customerId", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } }
])
```

Stages:

\$match → Filter documents

\$group → Group by customerId & sum amount

\$sort → Sort by total descending

Atomic Transactions? How Can We Achieve It in MongoDB

Atomic transactions allow multiple operations to be executed atomically (all succeed or all fail).
MongoDB supports multi-document transactions in replica sets.

```
const session = client.startSession();
session.startTransaction();

session.collection('users').updateOne({ _id: 1 }, { $inc: { balance: -100 } }, { session });
session.collection('accounts').updateOne({ _id: 2 }, { $inc: { balance: 100 } }, { session });

session.commitTransaction();
} catch {
  session.abortTransaction();
}
session.endSession();
```

Transactions Help in Rollback in Case of Failure

If any operation fails within a transaction, MongoDB rolls back all changes.
Ensures data consistency and prevents partial updates.
Example: Sending money → debit and credit must succeed together or rollback.

Difference Between Hashing, Encryption, and Encoding

Hashing	Encryption	Encoding
Verify integrity	Confidentiality	Data representation
❌ One-way	✅ Two-way	✅ Easily reversible
MD5, SHA256	AES, RSA	Base64, UTF-8
Password storage	Secure communication	Transfer over network

How to Identify Slow Queries

MongoDB:

```
useCollection.getProfilingStatus()
useSystem.profile.find().sort({ ts: -1 })
useIndex.plan: db.users.find({...}).explain("executionStats")
```

for COLLSCAN → indicates collection scan → needs indexing.

5. Replica Sets and Sharding Mechanism in MongoDB

Replica Set:

A group of MongoDB servers maintaining the same data for high availability

One primary, multiple secondaries

Sharding:

Distributes data across multiple servers to scale horizontally

Uses a shard key to split data

6. How to Configure Failover and Disaster Recovery in MongoDB

Replica sets provide automatic failover: if primary fails, secondary becomes primary.

Tools: Use mongodump or cloud snapshots

Disaster recovery: Multi-region replica sets, monitoring with Ops Manager or Atlas.

7. Load Balancer and How to Implement in Node.js

Load Balancer: Distributes incoming traffic across multiple Node.js instances to improve performance and availability
 Configuration Example (with Nginx):

```

upstream node_app {
    server 127.0.0.1:3000;
    server 127.0.0.1:3001;
}

server {
    listen 80;
    location / {
        proxy_pass http://node_app;
    }
}

```

Node.js + Cluster module + Nginx or HAProxy → scalable architecture

Benefits: High availability, fault tolerance, horizontal scaling.

8. How to Avoid Schema Pitfalls

Define clear schemas when using MongoDB (Mongoose recommended)

Specify required fields and validators to prevent inconsistent data:

```

const Schema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  age: { type: Number, min: 0 }
});

```

Common pitfalls avoidance:

Avoid embedding huge arrays in one document → use referencing

Create indexes carefully → avoid unnecessary overhead

Normalize vs denormalize data based on read/write patterns

9. Things to be Care Taken to Write Efficient Queries

Use explain() in MongoDB to see query execution plan:

```

const query = { age: { $gt: 18 } };
const plan = db.collection('users').explain("executionStats");

```

Warning: for COLLSCAN → indicates collection scan

Optimize by:

1. Create proper indexes

ing \$regex on unindexed fields
 ing returned fields (projection)
 aggregation pipelines for complex queries
 : Query execution time, number of documents scanned, and index usage.

o Implement Health Checks in MongoDB

h checks ensure your app can connect to DB before serving requests:

```
health', async (req, res) => {

mongoose.connection.db.admin().ping();
atus(200).json({ status: 'UP' });
(err) {
atus(503).json({ status: 'DOWN', error: err.message });
```

e extended to check:

cation lag in replica sets

y and storage usage

cy of key queries

re the SOLID Principles for Software Development

principles for maintainable and scalable code:

ingle Responsibility Principle

ss/module should have one reason to change

pen/Closed Principle

for extension, closed for modification

iskov Substitution Principle

pes must be replaceable with base types

nterface Segregation Principle

specific interfaces > one general-purpose interface

ependency Inversion Principle

d on abstractions, not concretions

n clean, reusable, testable Node.js code.

Is Kafka? Explain the Use Cases of Kafka

e Kafka: Distributed streaming platform for real-time data pipelines and messaging

omponents:

cers → Send messages

s → Logical channels

mers → Read messages

rs → Kafka servers

time analytics (user activity tracking)

sourcing for microservices

ng pipelines

ging system between services

ample: Using kafkajs

```
kafka } = require('kafkajs');
```

```
a = new Kafka({ clientId: 'app', brokers: ['localhost:9092'] });
```

```
ucer = kafka.producer();
```

```
ucer.connect();
```

```
ucer.send({ topic: 'logs', messages: [{ value: 'Hello Kafka' }] });
```

How do we minimize middleware overhead?

Best practices for Express middleware:

- Use middleware only to routes that need it:
- Avoid heavy synchronous operations inside middleware
- Use `async/await` with proper error handling to prevent blocking
- Use `app.use()` or chain related middleware instead of having many small ones
- Cache results when possible to avoid repeated processing
- Optimize to reduce request-response latency and CPU usage.

How do you ensure your team delivers high-quality code within deadlines?

We ensure quality and timely delivery by breaking down tasks into smaller, manageable sprints with clear goals. We maintain branching strategies like feature, develop, and main branches in Git, and use CI/CD tools (like GitHub Actions) and daily stand-up meetings help us track progress, remove blockers, and maintain accountability.

Describe a situation where you had to mentor a junior developer. How did you handle it?

When a junior developer struggled with modern web technologies, I guided a new developer who was struggling with React component structure and state management. I paired with them during code reviews, explained best practices like component reusability, props drilling vs context, and encouraged them to learn by doing – assigning smaller tasks, then gradually moving to complex features. Within a week, they were confident and contributing significantly.

Can you explain the SOLID principles and how you apply them in your designs?

How do you write maintainable and scalable code?

- Single Responsibility: Each component or function does one job (e.g., a Button component only handles UI, not logic).
- Open/Closed: Components are open for extension but closed for modification – for example, using props to extend functionality.
- Liskov Substitution: Derived components or classes should work as their parent – applicable in reusable UI components.
- Interface Segregation: Avoid forcing components to depend on unused props or functions.
- Dependency Inversion: Depend on abstractions, not concrete implementations – e.g., use custom hooks instead of direct API calls.

What are the design patterns in JavaScript?

Common JavaScript design patterns include:

- Module Pattern: Encapsulate code into reusable, self-contained modules.
- Singleton Pattern: Maintain a single instance (e.g., global state or config).
- Observer Pattern: Used in React's state management and event handling (e.g., Redux store subscribers).
- Factory Pattern: Create objects without specifying the exact class (useful in component creation).
- Decorator Pattern: Add new behaviors without modifying existing code – e.g., higher-order components (HOCs) in React.

How do you decide which tech stack is suitable for a specific project?

I consider the project's scale, performance needs, team expertise, and deployment target. For single-page apps, I prefer React + TypeScript + Redux + REST/GraphQL. For real-time or large apps, I choose Next.js (SSR/SSG) for SEO and performance. For enterprise apps, Angular can be better due to strict structure. I also consider developer community support, scalability, and integration needs before finalizing the stack.

What is the difference between Development, FT, UAT, Pre-prod, and Production environments?

Development: Used by developers to write and test code locally.
Functional Testing): QA team verifies the functional correctness of new features.
User Acceptance Testing): End users validate the app meets business requirements.
Staging: A mirror of production used for final checks before going live.
Production: The live environment where end users interact with the application.

How do you ensure 100% code coverage in your projects?

100% coverage is challenging but possible through a testing culture:
- Jest + React Testing Library for unit & component testing.
- Integrate coverage tools (like Istanbul or Coveralls) in CI pipelines.
- Test all edge cases, components, hooks, and user flows.
- Code reviews ensure every new feature includes matching test cases.
- Focus not only on numbers but also on meaningful coverage – testing logic and user behavior.

Can you explain the difference between Unit Testing, Integration Testing, and Functional Testing?

Unit Testing: Tests individual functions/components in isolation. (e.g., a React Button component)
Integration Testing: Tests how modules work together (e.g., form + API integration).
Functional Testing: Verifies that the app behaves as expected from the user's perspective (e.g., Cypress test).

How do you secure sensitive data in a MERN stack application?

Authentication: Never store tokens in localStorage; use HTTP-only cookies.
Communication: Use HTTPS, sanitize inputs, and validate user data.
Passwords: Hash passwords with bcrypt, use JWT for authentication, and apply CORS & Helmet for header security.
Environment Variables: Store secrets in .env files, never commit them.
Database Layer: Use parameterized queries to prevent injection attacks.

Describe your experience working in an Agile/Scrum environment.

2-week sprint cycles with daily stand-ups, sprint planning, and retrospectives.
Tasks tracked in JIRA, and each sprint includes coding, testing, and peer reviews.
Team members participate in story estimation, collaboration with UI/UX designers, and demo sessions to show progress.
Sprints ensure continuous delivery and quick feedback from stakeholders, improving overall product quality and

What are the key meetings in meetings like sprint planning, daily standup, backlog, sprint review, sprint retrospective

Sprint Planning: Team selects user stories from the backlog for the upcoming sprint and estimates effort.
Daily Standup: Short daily meeting where each member shares what they did yesterday, what they plan today, and any blockers.
Backlog: A prioritized list of features, bug fixes, and tasks to be completed in future sprints.
Sprint Review: Demonstrate completed features to stakeholders and gather feedback.
Sprint Retrospective: Reflect on the sprint to identify what went well, what didn't, and improvements for the next sprint.

How do you document your code and project architecture for your team?

Code: Maintain clear inline comments for complex logic.
README: Create README.md files for module explanations, setup instructions, and dependencies.
Architecture: Maintain architecture diagrams (e.g., component hierarchy, API flow, state management).
Reusable Components: Document reusable components, hooks, and utilities in a shared Confluence or Notion space.
UI Library: Maintain a component library or storybook for UI components documentation.

How do you onboard new developers to your team and projects?

Provide a project overview and access to repositories, documentation, and tools.
Define folder structure, coding standards, and workflows.
Assign small starter tasks to get familiar with code and processes.
Pair them with a mentor for code reviews and queries.
Conduct a demo session to explain architecture, API usage, and testing practices.

What tools do you use to track project progress and manage tasks?

Jira: Sprint planning, task assignment, and progress tracking.
Trello/ClickUp: Kanban boards for smaller tasks.
Slack/MS Teams: Team communication and updates.
GitHub/GitLab: Version control, PR reviews, and CI/CD pipelines.
Confluence/Notion: Documentation and knowledge sharing.

What are the key considerations when designing RESTful APIs?

Consistency: Standard endpoints and naming conventions.
Statelessness: Each request contains all info; server does not store session.
Versioning: Include version in URL (/api/v1/...) to manage updates.
Security: Authentication, authorization, and input validation.
Performance: Pagination, caching, and minimizing payload size.
Error Handling: Clear error messages and proper HTTP status codes.

What is your approach to deploying and scaling a MERN stack application?

Use CI/CD pipelines (GitHub Actions, Jenkins) for automated deployments.
Deploy frontend (React/Next.js) to Vercel/Netlify and backend (Node.js/Express) to AWS EC2/Elastic Beanstalk.
Use load balancers and horizontal scaling for backend services.
Implement database replication and caching (Redis/MongoDB Atlas) for performance.
Monitor for performance using New Relic, CloudWatch, or Grafana.

How would you handle a situation where the application is experiencing a memory leak?

Use Chrome DevTools memory profiler or Node.js heap snapshots to identify leaks.
Check for unreleased event listeners, large state objects, or circular references in React components.
Optimize by cleaning up subscriptions, timers, and context usage.
Use React.memo or useCallback to prevent unnecessary re-renders.
Restart the application and monitor after fixes.

What if a component you implemented in React is causing performance issues. How would you debug it?

Use React DevTools Profiler to identify slow components.
Check for excessive re-renders or heavy computations inside render.
Use memoization (React.memo, useMemo, useCallback) for expensive operations.
Optimize components using code-splitting and lazy loading.
Minimize API calls and minimize unnecessary state updates.

What steps would you take if your database suddenly becomes a bottleneck in your application?

Optimize queries using profiling tools to identify slow queries.

ment indexes, caching layers (Redis), and query optimization.
der read replicas for heavy read operations.
ate database scaling options: vertical (upgrade server) or horizontal (sharding).
e unnecessary joins or nested queries in frontend API calls.

ur team is stuck on a critical bug, how would you guide them to a solution?

duce the bug consistently to understand the issue.
e the problem into smaller components and debug individually.
ogging, breakpoints, and profiling tools.
rage peer brainstorming and pair programming for insights.
itimize fixing the bug and prepare a post-mortem to prevent recurrence.

is your approach to conducting code reviews?

e code readability, maintainability, and adherence to standards (ESLint/Prettier).
y proper component structure, reusability, and state management in React.
for performance optimizations (avoiding unnecessary re-renders, memoization).
w unit tests and edge cases coverage.
de constructive feedback with examples and encourage learning rather than just pointing out errors.
ve only after the code passes linting, testing, and functionality checks.

o you optimize MongoDB queries for better performance?

ndexes on frequently queried fields.
unnecessary full collection scans.
rojection to return only required fields.
ize queries with aggregation pipelines where needed.
der caching frequent queries using Redis.
or query performance using MongoDB profiler and explain().

o you manage error handling in an Express.js application?

ry-catch blocks in async functions.
ment centralized error-handling middleware to handle all errors consistently.
n standardized error responses with HTTP status codes and messages.
rrors with tools like Winston or Morgan for tracking.
ate operational errors vs programming errors for proper handling.

in how you would secure an Express.js API.

ment HTTPS to encrypt traffic.
WT or OAuth for authentication and authorization.
ate and sanitize all user inputs to prevent XSS, SQL Injection, or NoSQL injection.
CORS policies to restrict resource access.
elmet.js to set security headers.
request rate with rate-limiting middleware to prevent DDoS attacks.

o you optimize React applications for performance?

unnecessary re-renders using React.memo, useMemo, useCallback.
ment lazy loading and code splitting for large modules.

ize expensive operations in render functions.
irtualization for large lists (react-window/react-virtualized).
ize API calls and caching using React Query or SWR.
ess assets and optimize images.

o you secure a Node.js application?

sensitive data in environment variables, not in code.
ize inputs to prevent injection attacks.
elmet.js, CORS, and rate-limiting middleware.
passwords with bcrypt and use JWT/OAuth for authentication.
ate user sessions and implement token expiration.
arly update dependencies and monitor for vulnerabilities using npm audit.

ould you handle database sharding and replication in MongoDB?

cation: Set up primary-secondary replicas to ensure high availability. Automatic failover if primary fai
ing: Split collections across multiple shards using a shard key to handle large datasets.
or and optimize queries to distribute load evenly.
ne replication and sharding for scalable and fault-tolerant architecture.

ions of React in large-scale applications

t is powerful, it has some limitations in large applications:

ex State Management: For very large apps, managing state with only useState or useContext can become dif
rplate and Setup: For complex applications, setting up routing, state, and SSR can require significant b
rmance Overhead: Improper handling of components or excessive re-renders can hurt performance.
hallenges: React is client-side by default, which can impact SEO unless server-side rendering (Next.js)
ing Curve for Advanced Concepts: Hooks, Context, Suspense, Concurrent Mode, and Fiber can be overwhelmin
ration Complexity: Integrating React into legacy projects or with multiple frameworks can be tricky.

ct manages the Virtual DOM and its benefits

al DOM (VDOM) is an in-memory representation of the actual DOM. React maintains this virtual tree of ele
t works:

ment renders → React creates a virtual DOM tree.
ate or props change → a new VDOM tree is generated.
diffs the new tree against the previous VDOM tree.
the minimal set of changes are applied to the real DOM.

rmance: Minimizes expensive real DOM updates.

ctability: UI updates are declarative and controlled.

-platform rendering: Same VDOM logic works for web, native (React Native), or VR.

cance of React Fiber architecture

is a reimplementation of React's core algorithm (reconciler) introduced in React 16.

ignificance:

mental rendering: Breaks rendering work into units and spreads them over multiple frames.

itization: Higher priority updates (like user input) are processed before lower priority ones (like netw

rrency: Enables React to pause and resume work, improving responsiveness in large apps.

r error handling: Allows features like error boundaries.

ntially allows React to handle large, complex applications more smoothly by optimizing rendering and use

reconciliation process and efficient DOM updates

ciliation is how React updates the DOM efficiently.

ss:

a component updates, React compares the new VDOM with the previous VDOM (diffing).

calculates the minimal set of changes (patches).

updates only the necessary DOM nodes (instead of re-rendering everything).

iciency techniques:

elements: Helps React track list items efficiently.

updates: Multiple state updates are combined to reduce DOM operations.

omponent / React.memo: Prevent unnecessary renders of components.

ferences between functional and class components

unctional Components Class Components

unction-based, simpler ES6 class-based, verbose

gement useState and other hooks this.state and this.setState()

methods useEffect, useLayoutEffect componentDidMount, componentDidUpdate, componentWillUnmount

e Slightly faster, less boilerplate Slightly heavier due to this binding and class overhead

ort Fully supported Cannot use hooks directly (pre-hooks)

y & Testing Easier to test and maintain Slightly more complex, especially with inheritance

y support Fully compatible with concurrent mode Compatible but not recommended for new apps

Functional components with hooks are now the recommended approach for new React applications due to simp

t, I can also make a diagram showing Virtual DOM → Fiber → Reconciliation which will make it super easy
t me to do that?

ks fully replace Redux? Why or why not? 7. What are best practices for managing state in large React app
id:

etailed breakdown for your next set of React questions:

ks fully replace Redux? Why or why not?

t completely. Hooks (like useState, useReducer, useContext) can manage state locally or even globally, b

of Hooks for state:

ate and useReducer handle component-level state effectively.

ntext allows global state without extra libraries.

boilerplate compared to Redux.

ations compared to Redux:

ery large applications, useContext can cause unnecessary re-renders of all consumers on state change.

provides time-travel debugging, middleware (like redux-thunk / redux-saga), and a predictable centraliz
eware support and dev-tools are robust in Redux.

ooks can replace Redux in medium or small apps, but for large-scale apps with complex state flows, Redux

actices for managing state in large React apps

ocal state where possible (useState) and lift state up only when necessary.

l state wisely: Use useContext or libraries like Redux, Zustand, or Recoil for global/shared state.

unnecessary context usage: Only wrap components that actually need the context to prevent extra re-rend

lize data: For large datasets, store them in normalized shape (like a database) to simplify updates.

emoization (React.memo, useMemo, useCallback) for expensive computations.

state logically: Don't put all state in one big object; separate concerns (UI state vs. domain state).

educers for complex state (useReducer), especially for nested objects or arrays.

able updates: Always update state immutably to avoid subtle bugs.

prevent unnecessary re-renders

- `React.memo` for functional components to memoize output unless props change.
- `useMemo` to memoize expensive computed values.
- `useCallback` to memoize functions passed as props.
- `React.Fragment` components: Break large components into smaller ones so that only affected parts re-render.
- `React.memo` anonymous functions in JSX when possible (they trigger new references every render).
- `React.memo` context usage: Only include necessary values in context to avoid triggering unnecessary re-renders.

```

nces between useMemo() and useCallback()
seMemo()    useCallback()
emoizes computed values    Memoizes functions
alue    Function
xpensive calculation caching    Prevent function recreation on re-render
const memoValue = useMemo(() => computeHeavyValue(a, b), [a, b]);    const memoFunc = useCallback(() => do
umb: Use useMemo for values, use useCallback for functions passed to child components or dependencies.

```

se of `useEffect()` for fetching data

`useEffect()` is commonly used to fetch data on component mount or when dependencies change.

le workflow:

```
import { useEffect, useState } from "react";
```

```
function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
```

```
Effect(() => {
  sync function fetchUsers() {
    const response = await fetch("https://api.example.com/users");
    const data = await response.json();
    setUsers(data);
    setLoading(false);
  }
});
```

```
(loading) return <div>Loading...</div>;  
return <ul>{users.map(user => <li key={user.id}>{user.name}</li>)}</ul>;
```

```
:
fect with empty dependencies runs once (like componentDidMount).
up: You can cancel ongoing requests in cleanup function to prevent memory leaks.
lso run on dependency change to fetch data dynamically.
```

t, I can also make a diagram showing how useEffect, useMemo, useCallback, and React.memo interact for performance. Would you like me to make that diagram?

How would you optimize large component trees? 12. How would you optimize performance for large lists or grids?

detailed explanation for your next set of React performance questions:

Could you optimize large component trees?
component trees, unnecessary re-renders can kill performance. Optimization strategies:
react.memo

ze components so they only re-render when props change.

- down components
- large components into smaller, focused components to localize updates.
- unnecessary state in parent components
- state as close as possible to where it is used.
- seCallback and useMemo
- ze functions and expensive calculations to prevent child re-renders.
- alize components
- r only visible components for large trees (e.g., using libraries like react-window or react-virtualized)
- load subtrees
- react.lazy + Suspense to load non-critical parts on demand.
- ontext wisely
 - putting frequently changing values in Context; it triggers re-render of all consumers.

ould you optimize performance for large lists or grids?

thousands of items directly is slow. Strategies:

- alization
 - r only the items visible in the viewport. Libraries: react-window, react-virtualized.
- rops
 - s provide unique keys to list items to help React identify changes efficiently.
 - inline functions and objects
 - nt new references on every render, which can trigger re-renders of child components.
- react.memo for list items
- ze individual list items so only changed items re-render.
- ation / Infinite scroll
 - data in chunks to reduce initial render load.
 - updates
 - multiple state updates together to reduce re-renders.

s lazy loading, and how does it help?

ng is a technique where components or resources are loaded only when needed, rather than all at once.

mented using React.lazy() + Suspense.

- split large bundles into smaller chunks and improves initial load time.

```
import React, { Suspense } from 'react';
```

```
const LargeComponent = React.lazy(() => import('./LargeComponent'));
```

```
function App() {
```

```
  <h1>My App</h1>
```

```
  <Suspense fallback=<div>Loading...</div>>
```

```
    <LargeComponent />
```

```
  </Suspense>
```

r initial page load.

es unused code in the initial bundle.

ves perceived performance for users.

t, I can also make a performance optimization checklist for React apps, summarizing Virtual DOM, Fiber, t me to make that?

es React handle side effects, and how can you manage them? 15. How would you handle errors in React, and id:

etailed explanation for your next set of React questions:

es React handle side effects, and how can you manage them?

ts in React are operations that affect things outside the component, such as:

ing data from an API

ribing to events

s (setTimeout, setInterval)

ulating the DOM directly

ide Effects:

les side effects primarily with the useEffect hook in functional components, or lifecycle methods in cla

th useEffect:

seEffect, useState } from 'react';

imer() {

ount, setCount] = useState(0);

t(() => {

interval = setInterval(() => setCount(c => c + 1), 1000);

anup function to prevent memory leaks

() => clearInterval(interval);

// Empty dependency array → runs only on mount

div>Timer: {count}</div>;

:

dencies array controls when the effect runs.

run once on mount.

→ run whenever dep changes.

up prevents memory leaks and dangling subscriptions.

uld you handle errors in React, and what is the role of error boundaries?

ling in React:

atch in event handlers – handle synchronous errors.

se .catch() – handle async errors (like API calls).

boundaries – handle rendering errors in component trees.

daries:

components that catch JavaScript errors anywhere in their child component tree, log them, and display a

mented using class components with lifecycle methods:

ct from 'react';

rBoundary extends React.Component {

tor(props) {

props);

tate = { hasError: false };

```

    getDerivedStateFromError(error) {
      // Update state so the error will be shown
      // as fallback UI
      return { hasError: true };
    }

    componentDidCatch(error, errorInfo) {
      // You can use your own logging to help
      // find errors in your application.
      // e.g. log(error, errorInfo);
    }

    render() {
      if (this.state.hasError) {
        // You can render any custom fallback UI
        return <h1>Something went wrong.</h1>;
      }

      return this.props.children;
    }
  }
}

export default ErrorBoundary;

```

class components can be error boundaries (functional components cannot, but libraries like `react-error-boundary` can). Error boundaries catch JavaScript errors in child components and display fallback UIs.

Error boundaries are useful for critical UI sections like dashboards or forms.